PHP Unit 2

PHP Form Handling

The PHP superglobals $_GET and $_POST are used to collect form-data.

PHP - A Simple HTML Form

The example below displays a simple HTML form with two input fields and a submit button:

Example

```
<html>
<body>

<form action="welcome.php"
method="post"> Name: <input
type="text" name="name"><br> E-
mail: <input type="text"
name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

When the user fills out the form above and clicks the submit button, the form data is sent for processing to a PHP file named "welcome.php". The form data is sent with the HTTP POST method.

To display the submitted data you could simply echo all the variables. The "welcome.php" looks like this:

```
<html>
<body>

Welcome <?php echo $_POST["name"]; ?><br>
Your email address is: <?php echo $_POST["email"]; ?>

</body>
</html>
```

The output could be something like

this: Welcome John

Ashwani Mishra

Your email address is john.doe@example.com

The same result could also be achieved using the HTTP GET method:

Example

```
<html>
<body>

<form action="welcome_get.php"
method="get"> Name: <input type="text"
name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

and "welcome_get.php" looks like this:

```
<html>
<body>

Welcome <?php echo $_GET["name"]; ?><br>
Your email address is: <?php echo $_GET["email"]; ?>

</body>
</html>
```

The code above is quite simple. However, the most important thing is missing. You need to validate form data to protect your script from malicious code.

GET vs. POST

Both GET and POST create an array (e.g. array(key1 => value1, key2 => value2, key3 => value3, ...)). This array holds key/value pairs, where keys are the names of the form controls and values are the input data from the user.

Both GET and POST are treated as $_GET and $_POST. These are superglobals, which means that they are always accessible, regardless of scope - and you can access them from any function, class or file without having to do anything special.

$_GET is an array of variables passed to the current script via the URL parameters.

$_POST is an array of variables passed to the current script via the HTTP POST method.

Ashwani Mishra

When to use GET?

Information sent from a form with the GET method is visible to everyone (all variable names and values are displayed in the URL). GET also has limits on the amount of information to send. The limitation is about 2000 characters. However, because the variables are displayed in the URL, it is possible to bookmark the page. This can be useful in some cases.

GET may be used for sending non-sensitive data.

Note: GET should NEVER be used for sending passwords or other sensitive information!

When to use POST?

Information sent from a form with the POST method is invisible to others (all names/values are embedded within the body of the HTTP request) and has no limits on the amount of information to send.

Moreover POST supports advanced functionality such as support for multi-part binary input while uploading files to server.

However, because the variables are not displayed in the URL, it is not possible to bookmark the page. Developers prefer POST for sending form data.

Environment Variables

$_ENV

$HTTP_ENV_VARS [deprecated]

- *(PHP 4 >= 4.1.0, PHP 5, PHP 7)*
- *$_ENV -- $HTTP_ENV_VARS [deprecated] — Environment variables*
- *An associative array of variables passed to the current script via the environment method.*
- *These variables are imported into PHP's global namespace from the environment under which the PHP parser is running. Many are provided by the shell under which PHP is running and different systems are likely running different kinds of shells, a definitive list is impossible. Please see your shell's documentation for a list of defined environment variables.*
- *Other environment variables include the CGI variables, placed there regardless of whether PHP is running as a server module or CGI processor.*
- *$HTTP_ENV_VARS contains the same initial information, but is not a superglobal. (Note that $HTTP_ENV_VARS and $_ENV are different variables and that PHP handles them as such)*

Ashwani Mishra

Example #1 $_ENV example

```
<?php
echo 'My username is ' .$_ENV["USER"] . '!';
?>
```

- *If $_ENV is empty because variables_order does not include it, it will be filled with values fetched by getenv().*
- *For example, when calling getenv("REMOTE_ADDR"), $_ENV['REMOTE_ADDR'] will be defined as well (if such an environment variable exists)*

PHP File Handling

File handling is an important part of any web application. You often need to open and process a file for different tasks.

PHP Manipulating Files

PHP has several functions for creating, reading, uploading, and editing files.

Be careful when manipulating files!

When you are manipulating files you must be very careful.

You can do a lot of damage if you do something wrong. Common errors are: editing the wrong file, filling a hard-drive with garbage data, and deleting the content of a file by accident.

PHP readfile() Function

The readfile() function reads a file and writes it to the output

buffer. Assume we have a text file called "webdictionary.txt

The PHP code to read the file and write it to the output buffer is as follows (the readfile() function returns the number of bytes read on success):

Example

```
<?php
echo readfile("webdictionary.txt");
?>
```

Ashwani Mishra

PHP File

Open/Read/Close HP

Open File - fopen()

A better method to open files is with the fopen() function. This function gives you more options than the readfile() function.

We will use the text file, "webdictionary.txt",

The first parameter of fopen() contains the name of the file to be opened and the second parameter specifies in which mode the file should be opened. The following example also generates a message if the fopen() function is unable to open the specified file:

Example

```
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to
open file!"); echo fread($myfile,filesize("webdictionary.txt"));
fclose($myfile);
?>
```

Tip: The fread() and the fclose() functions will be explained

below. The file may be opened in one of the following modes:

Modes	Description
r	Open a file for read only. File pointer starts at the beginning of the file
w	Open a file for write only. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a	Open a file for write only. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x	Creates a new file for write only. Returns FALSE and an error if file already exists
r+	Open a file for read/write. File pointer starts at the beginning of the file

Shivani Mishra

w+	Open a file for read/write. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a+	Open a file for read/write. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x+	Creates a new file for read/write. Returns FALSE and an error if file already exists

PHP Read File - fread()

The fread() function reads from an open file.

The first parameter of fread() contains the name of the file to read from and the second parameter specifies the maximum number of bytes to read.

The following PHP code reads the "webdictionary.txt" file to the end:

```
fread($myfile,filesize("webdictionary.txt"));
```

PHP Close File - fclose()

The fclose() function is used to close an open file.

It's a good programming practice to close all files after you have finished with them. You don't want an open file running around on your server taking up resources!

The fclose() requires the name of the file (or a variable that holds the filename) we want to close:

```
<?php
$myfile = fopen("webdictionary.txt", "r");
// some code to be executed....
fclose($myfile);
?>
```

PHP Read Single Line - fgets()

The fgets() function is used to read a single line from a file.

The example below outputs the first line of the "webdictionary.txt" file:

Ashwani Mishra

Example

```
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to
open file!"); echo fgets($myfile);
fclose($myfile);
?>
```

Note: After a call to the fgets() function, the file pointer has moved to the next line.

PHP Check End-Of-File - feof()

The feof() function checks if the "end-of-file" (EOF) has been reached.

The feof() function is useful for looping through data of unknown length.

The example below reads the "webdictionary.txt" file line by line, until end-of-file is reached:

Example

```
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
// Output one line until end-of-file
while(!feof($myfile)) {
  echo fgets($myfile) . "<br>";
}
fclose($myfile);
?>
```

PHP Read Single Character - fgetc()

The fgetc() function is used to read a single character from a file.

The example below reads the "webdictionary.txt" file character by character, until end-of-file is reached:

Example

```
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
// Output one character until end-of-
file while(!feof($myfile)) {
  echo fgetc($myfile);
}
fclose($myfile);
?>
```

Note: After a call to the fgetc() function

Ashwani Mishra

PHP Directory Functions

The directory functions allow you to retrieve information about directories and their contents.

The PHP directory functions are part of the PHP core. No installation is required to use these functions.

PHP Directory Functions

Function	*Description*
chdir()	*Changes the current directory*
chroot()	*Changes the root directory*
closedir()	*Closes a directory handle*
dir()	*Returns an instance of the Directory class*
getcwd()	*Returns the current working directory*
opendir()	*Opens a directory handle*
readdir()	*Returns an entry from a directory handle*
rewinddir()	*Resets a directory handle*
scandir()	*Returns an array of files and directories of a specified directory*

PHP chdir() Function

The chdir() function changes the current directory.

Syntax

chdir(directory)

Example

Change the current directory:

```
<?php
// Get current directory
echo getcwd() .
"<br>";

// Change
directory
chdir("images");

// Get current directory
echo getcwd();
?>
```

readdir() Function

The readdir() function returns the name of the next entry in a directory.

Syntax

readdir(dir)

List all entries in the images directory, then close:

```
<?php
$dir = "/images/";

// Open a directory, and read its
contents  if (is_dir($dir)){
  if ($dh = opendir($dir)){
    while (($file = readdir($dh)) !==
      false){ echo "filename:" . $file .
      "<br>";
    }
    closedir($dh);
  }
}
?>
```

Result:

filename: cat.gif
filename: dog.gif
filename:

Ashwani Mishra

horse.gif

Ashwani Mishra

Identifying Data Origin

Use $_SERVER['HTTP_REFERER']. It is the address of the page (if any) which referred the user agent to the current page. This is set by the user agent. Not all user agents will set this, and some provide the ability to modify HTTP_REFERER as a feature.
For further restrictions you can perform the following. example.com should be changed to your domain.

if you want to check for more info, use something like the following snippet

```
$allowed = array('domain1', 'domain2', 'domain3');

if(isset($_SERVER['HTTP_ORIGIN']) && in_array($_SERVER['HTTP_ORIGIN'], $allowed)){
   // SELECT credentials for this user account from
   database if(isset($_GET['api_key'], $_GET['app_secret'])
      && $_GET['api_key'] == 'api_key_from_db'
      && $_GET['app_secret'] == 'app_secret_from_db'
   ){
      // all fine
   }else{
      // not allowed
   }
}else{
  // not allowed
 }
```

If the users have to pass more data to your service, use POST instead of GET

This returns an array so the Origin header should be available like this:

```
$request_headers = getallheaders();
$origin = $request_headers['Origin'];
```

PHP Data Validation

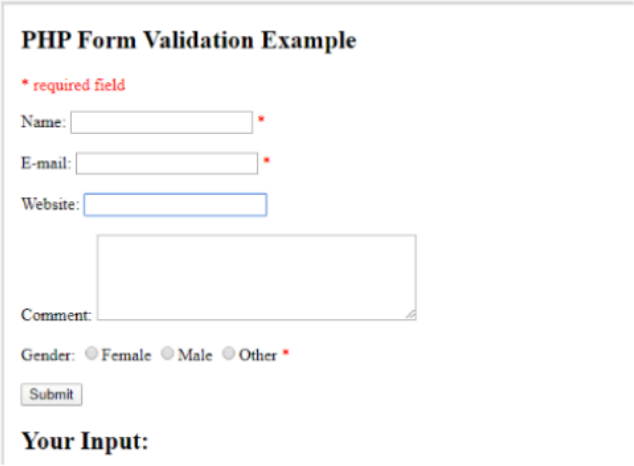
PHP Form Validation

Think SECURITY when processing PHP forms!

These pages will show how to process PHP forms with security in mind. Proper validation of form data is important to protect your form from hackers and spammers!

The HTML form we will be working at in these chapters, contains various input fields: required and optional text fields, radio buttons, and a submit button:

Vishwani Mishra



The validation rules for the form above are as follows:

Field	***Validation Rules***
Name	***Required. + Must only contain letters and whitespace***
E-mail	***Required. + Must contain a valid email address (with @ and .)***
Website	***Optional. If present, it must contain a valid URL***
Comment	***Optional. Multi-line input field (textarea)***
Gender	***Required. Must select one***

First we will look at the plain HTML code for the form:

The Form Element

The HTML code of the form looks like this:

```
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

When the form is submitted, the form data is sent with method="post".

What is the $_SERVER["PHP_SELF"] variable?

The $_SERVER["PHP_SELF"] is a super global variable that returns the filename of the currently executing script.

So, the $_SERVER["PHP_SELF"] sends the submitted form data to the page itself, instead of jumping to a different page. This way, the user will get error messages on the same page as the form.

What is the htmlspecialchars() function?

The htmlspecialchars() function converts special characters to HTML entities. This means that it will replace HTML characters like < and > with < and >. This prevents attackers from exploiting the code by injecting HTML or Javascript code (Cross-site Scripting attacks) in forms.

Big Note on PHP Form Security

The $_SERVER["PHP_SELF"] variable can be used by hackers!

If PHP_SELF is used in your page then a user can enter a slash (/) and then some Cross Site Scripting (XSS) commands to execute.

Cross-site scripting (XSS) is a type of computer security vulnerability typically found in Web applications. XSS enables attackers to inject client-side script into Web pages viewed by other users.

Assume we have the following form in a page named "test_form.php":

```
<form method="post" action="<?php echo $_SERVER["PHP_SELF"];?>">
```

Now, if a user enters the normal URL in the address bar like "http://www.example.com/test_form.php", the above code will be translated to:

```
<form method="post"

action="test_form.php">
```

So far, so good.

However, consider that a user enters the following URL in the address bar:

http://www.example.com/test_form.php/%22%3E%3Cscript%3Ealert('hacked')%3C/script

%3E In this case, the above code will be translated to:

```
<form method="post" action="test_form.php/"><script>alert('hacked')</script>
```

This code adds a script tag and an alert command. And when the page loads, the JavaScript code will be executed (the user will see an alert box). This is just a simple and harmless example how the PHP_SELF variable can be exploited.

Be aware of that any JavaScript code can be added inside the <script> tag! A hacker can redirect the user to a file on another server, and that file can hold malicious code that can alter the global variables or submit the form to another address to save the user data, for example.

Ashwani Mishra

How To Avoid $_SERVER["PHP_SELF"] Exploits?

$_SERVER["PHP_SELF"] exploits can be avoided by using the htmlspecialchars()

function. The form code should look like this:

```
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

The htmlspecialchars() function converts special characters to HTML entities. Now if the user tries to exploit the PHP_SELF variable, it will result in the following output:

```
<form  method="post"

action="test_form.php/&quot;&gt;&lt;script&gt;alert('hacked')&lt;/script&gt;">
```

The

exploit attempt fails, and no harm is done!

Validate Form Data With PHP

The first thing we will do is to pass all variables through PHP's htmlspecialchars() function.

When we use the htmlspecialchars() function; then if a user tries to submit the following in a text field:

<script>location.href('http://www.hacked.com')</script>

- this would not be executed, because it would be saved as HTML escaped code,

like this: <script>location.href('http://www.hacked.com')</script>

The code is now safe to be displayed on a page or inside an e-

mail. We will also do two more things when the user submits

the form:

1. Strip unnecessary characters (extra space, tab, newline) from the user input data (with the PHP trim() function)
2. Remove backslashes (\) from the user input data (with the PHP stripslashes() function)

The next step is to create a function that will do all the checking for us (which is much more convenient than writing the same code over and over again).

We will name the function test_input().

Now, we can check each $_POST variable with the test_input() function, and the script looks like this:

Example

Ashwani Mishra

```
<?php
// define variables and set to empty values
$name = $email = $gender = $comment =

$website = ""; if

($_SERVER["REQUEST_METHOD"] ==

"POST") {
```

Ashwani Mishra

```
  $name = test_input($_POST["name"]);
  $email = test_input($_POST["email"]);
  $website = test_input($_POST["website"]);
  $comment = test_input($_POST["comment"]);
  $gender = test_input($_POST["gender"]);
}

function test_input($data) {
  $data = trim($data);
  $data = stripslashes($data);
  $data =
  htmlspecialchars($data);
  return $data;
}
?>
```

How to make a redirect in PHP?

Redirection from one page to another in PHP is commonly achieved using the following two ways: Using Header Function in PHP:
The header() function is an inbuilt function in PHP which is used to send the raw HTTP (Hyper Text Transfer Protocol) header to the client.

Syntax

```
header( $header, $replace, $http_response_code )
```

Parameters: This function accepts three parameters as mentioned above and described below:

- ***$header: This parameter is used to hold the header string.***
- ***$replace: This parameter is used to hold the replace parameter which indicates the header should replace a previous similar header, or add a second header of the same type. It is optional parameter.***
- ***$http_response_code: This parameter hold the HTTP response code. Below program illustrates the header() function in PHP:***

Program:

```
<?php

// Redirect browser
header("Location: http://www.geeksforgeeks.org");

exit;
?>
```

PHP Cookies

What is a Cookie?

A cookie is often used to identify a user. A cookie is a small file that the server embeds on the user's computer. Each time the same computer requests a page with a browser, it will send the cookie too. With PHP, you can both create and retrieve cookie values.

Ashwani Mishra

Create Cookies With PHP

A cookie is created with the setcookie() function.

Syntax

setcookie(name, value, expire, path, domain, secure, httponly);

Only the name parameter is required. All other parameters are optional.

PHP Create/Retrieve a Cookie

The following example creates a cookie named "user" with the value "John Doe". The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website (otherwise, select the directory you prefer).

We then retrieve the value of the cookie "user" (using the global variable $_COOKIE). We also use the isset() function to find out if the cookie is set:

Example

```
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 day
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_na
me])) {
   echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
   echo "Cookie '" . $cookie_name . "' is
   set!<br>"; echo "Value is: " .
   $_COOKIE[$cookie_name];
}
?>

</body>
</html>
```

Modify a Cookie Value

To modify a cookie, just set (again) the cookie using the setcookie() function:

Ashwani Mishra

Example

```
<?php
$cookie_name = "user";
$cookie_value = "Alex Porter";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/");
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_na
me])) {
  echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
  echo "Cookie '" . $cookie_name . "' is
  set!<br>"; echo "Value is: " .
  $_COOKIE[$cookie_name];
}
?>

</body>
</html>
```

Delete a Cookie

To delete a cookie, use the setcookie() function with an expiration date in the past:

Example

```
<?php
// set the expiration date to one hour
ago setcookie("user", "", time() - 3600);
?>
<html>
<body>

<?php
echo "Cookie 'user' is deleted.";
?>

</body>
</html>
```

Ashwani Mishra

PHP Sessions

A session is a way to store information (in variables) to be used across multiple

pages. Unlike a cookie, the information is not stored on the users computer.

What is a PHP Session?

When you work with an application, you open it, do some changes, and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are or what you do, because the HTTP address doesn't maintain state.

Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc). By default, session variables last until the user closes the browser.

So; Session variables hold information about one single user, and are available to all pages in one application.

Tip: If you need a permanent storage, you may want to store the data in a database.

Start a PHP Session

A session is started with the session_start() function.

Session variables are set with the PHP global variable: $_SESSION.

Now, let's create a new page called "demo_session1.php". In this page, we start a new PHP session and set some session variables:

Example

```
<?php
// Start the
session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] =
"cat"; echo "Session variables
are set.";
?>
```

Ashwani Mishra

```
</body>
</html>
```

Get PHP Session Variable Values

Next, we create another page called "demo_session2.php". From this page, we will access the session information we set on the first page ("demo_session1.php").

Notice that session variables are not passed individually to each new page, instead they are retrieved from the session we open at the beginning of each page (session_start()).

Also notice that all session variable values are stored in the global $_SESSION variable:

Example

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Echo session variables that were set on
previous page echo "Favorite color is " .
$_SESSION["favcolor"] . ".<br>"; echo "Favorite
animal is " . $_SESSION["favanimal"] . ".";
?>

</body>
</html>
```

Another way to show all the session variable values for a user session is to run the following code:

Example

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
print_r($_SESSI
ON);
?>

</body>
```

Ashwani Mishra

```
</html>
```

Ashwani Mishra

PHP Mail Functions

PHP Mail Introduction

The mail() function allows you to send emails directly from a script.

Requirements

For the mail functions to be available, PHP requires an installed and working email system. The program to be used is defined by the configuration settings in the php.ini file.

Installation

The mail functions are part of the PHP core. There is no installation needed to use these functions.

Runtime Configuration

The behavior of the mail functions is affected by settings in php.ini:

Name	Default	Description	Changeable
mail.add_x_header	"0"	Add X-PHP-Originating-Script that will include UID of the script followed by the filename. For PHP 5.3.0 and above	PHP_INI_PERDIR
mail.log	NULL	The path to a log file that will log all mail() calls. Log include full path of script, line number, To address and headers. For PHP 5.3.0 and above	PHP_INI_PERDIR
SMTP	"localhost"	Windows only: The DNS name or IP address of the SMTP server	PHP_INI_ALL
smtp_port	"25"	Windows only: The SMTP port number. For PHP 4.3.0 and above	PHP_INI_ALL

Ashwani Mishra

sendmail_from	NULL	Windows only: Specifies the "from" address to be used when sending mail from mail()	PHP_INI_ALL
sendmail_path	"/usr/sbin/sendmail -t -i"	Specifies where the sendmail program can be found. This directive works also under Windows. If set, SMTP, smtp_port and sendmail_from are ignored	PHP_INI_SYSTEM

PHP mail() Function

Example

Send a simple email:

```
<?php
// the message
$msg = "First line of text\nSecond line of text";

// use wordwrap() if lines are longer than 70 characters
$msg = wordwrap($msg,70);

// send email
mail("someone@example.com","My subject",$msg);
?>
```

The mail() function allows you to send emails directly from a script.

Syntax

mail(to,subject,message,headers,param

eters);

Parameter Values

Parameter	Description
to	Required. Specifies the receiver / receivers of the email
subject	Required. Specifies the subject of the email. Note: This parameter cannot contain any newline characters

Ashwani Mishra

message	***Required. Defines the message to be sent. Each line should be separated with a LF (\n). Lines should not exceed 70 characters.*** ***Windows note: If a full stop is found on the beginning of a line in the message, it might be removed. To solve this problem, replace the full stop with a double dot:*** ***<?php*** ***$txt = str_replace("\n.", "\n..", $txt);*** ***?>***
headers	***Optional. Specifies additional headers, like From, Cc, and Bcc. The additional headers should be separated with a CRLF (\r\n).*** ***Note: When sending an email, it must contain a From header. This can be set with this parameter or in the php.ini file.***
parameters	***Optional. Specifies an additional parameter to the sendmail program (the one defined in the sendmail_path configuration setting). (i.e. this can be used to set the envelope sender address when using sendmail with the -f sendmail option)***

More Examples

Send an email with extra headers:

```
<?php
$to = "somebody@example.com";
$subject = "My subject";
$txt = "Hello world!";
$headers = "From: webmaster@example.com"
. "\r\n" . "CC: somebodyelse@example.com";

mail($to,$subject,$txt,$headers);
?>
```

PHP What is OOP?

OOP stands for Object-Oriented Programming.

Procedural programming is about writing procedures or functions that perform operations on the data, while object-oriented programming is about creating objects that contain both data and functions.

Object-oriented programming has several advantages over procedural programming:

- ***OOP is faster and easier to execute***

Ashwani Mishra

- *OOP provides a clear structure for the programs*
- *OOP helps to keep the PHP code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug*
- *OOP makes it possible to create full reusable applications with less code and shorter development time*

PHP OOP - Classes and Objects

A class is a template for objects, and an object is an instance of class.

OOP Case

Let's assume we have a class named Fruit. A Fruit can have properties like name, color, weight, etc. We can define variables like $name, $color, and $weight to hold the values of these properties.

When the individual objects (apple, banana, etc.) are created, they inherit all the properties and behaviors from the class, but each object will have different values for the properties.

Define a Class

A class is defined by using the class keyword, followed by the name of the class and a pair of curly braces ({}). All its properties and methods goes inside the braces:

Syntax

```
<?php
class
Fruit {
  // code goes here...
}
?>
```

Below we declare a class named Fruit consisting of two properties ($name and $color) and two methods set_name() and get_name() for setting and getting the $name property:

Example

```
<?php
class
Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function  set_name($name)  {
    $this->name = $name;
```

Ashwani Mishra

```
}
```

Ashwani Mishra

```
  function get_name() {
    return $this->name;
  }
}
?>
```

Define Objects

Classes are nothing without objects! We can create multiple objects from a class. Each object has all the properties and methods defined in the class, but they will have different property values.

Objects of a class is created using the new keyword.

In the example below, $apple and $banana are instances of the class Fruit:

Example

```
<?php
class
Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function  set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}

$apple = new Fruit();
$banana = new Fruit();
$apple->set_name('Apple');
$banana->set_name('Banana');

echo $apple-
>get_name(); echo
"<br>";
echo $banana->get_name();
?>
```

n the example below, we add two more methods to class Fruit, for setting and getting the $color property:

Example

Ashwani Mishra

```php
<?php
```

Ashwani Mishra

```
class Fruit {
 // Properties
 public $name;
 public $color;

 // Methods
 function set_name($name) {
  $this->name = $name;
 }
 function get_name() {
  return $this->name;
 }
 function set_color($color) {
  $this->color = $color;
 }
 function get_color() {
  return $this->color;
 }
}

$apple = new Fruit();
$apple->set_name('Apple');
$apple->set_color('Red');
echo "Name: " . $apple-
>get_name(); echo "<br>";
echo "Color: " . $apple->get_color();
?>
```

PHP - The $this Keyword

The $this keyword refers to the current object, and is only available inside

methods. Look at the following example:

Example

```
<?php
class Fruit
{
 public $name;
}
$apple = new Fruit();
?>
```

So, where can we change the value of the $name property? There are two ways:

1. Inside the class (by adding a set_name() method and use $this):

Ashwani Mishra

Example

```
<?php
class
Fruit {
  public $name;
  function set_name($name) {
    $this->name = $name;
  }
}
$apple = new Fruit();
$apple->set_name("Apple");
?>
```

2. Outside the class (by directly change the property value):

Example

```
<?php
class
Fruit {
  public $name;
}
$apple = new Fruit();
$apple->name = "Apple";
?>
```

PHP - instanceof

You can use the instanceof keyword to check if an object belongs to a specific class:

Example

```
<?php
$apple = new Fruit();
var_dump($apple instanceof
Fruit);
?>
```

PHP Functions

The real power of PHP comes from its functions.

PHP has more than 1000 built-in functions, and in addition you can create your own custom functions.

PHP Built-in Functions

PHP has over 1000 built-in functions that can be called directly, from within a script, to perform a specific task.

Ashwani Mishra

Please check out our PHP reference for a complete overview of the PHP built-in functions.

Ashwani Mishra

PHP User Defined Functions

Besides the built-in PHP functions, it is possible to create your own functions.

- ***A function is a block of statements that can be used repeatedly in a program.***
- ***A function will not execute automatically when a page loads.***
- ***A function will be executed by a call to the function.***

Create a User Defined Function in PHP

A user-defined function declaration starts with the word function:

Syntax

```
function  functionName()  {
   code to be executed;
}
```

Note: A function name must start with a letter or an underscore. Function names are NOT case-sensitive.

Tip: Give the function a name that reflects what the function does!

In the example below, we create a function named "writeMsg()". The opening curly brace ({) indicates the beginning of the function code, and the closing curly brace (}) indicates the end of the function.
The function outputs "Hello world!". To call the function, just write its name followed by brackets ():

Example

```
<?php
function
   writeMsg() {
   echo "Hello
   world!";
}

writeMsg(); // call the function
?>
```

PHP Function Arguments

Information can be passed to functions through arguments. An argument is just like a variable.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument ($fname). When the familyName() function is called, we also pass along a name (e.g. Jani), and the name is used inside the function, which outputs several different first names, but an equal last name:

Ashwani Mishra

Example

```
<?php
function familyName($fname)
  { echo "$fname
  Refsnes.<br>";
}

familyName("Jani");
familyName("Hege");
familyName("Stale");
familyName("Kai Jim");
familyName("Borge");
?>
```

PHP Functions - Returning values

To let a function return a value, use the return statement:

Example

```
<?php declare(strict_types=1); // strict
requirement function sum(int $x, int $y) {
  $z = $x + $y;
  return $z;
}

echo "5 + 10 = " . sum(5, 10) . "<br>";
echo "7 + 13 = " . sum(7, 13) .
"<br>"; echo "2 + 4 = " . sum(2,
4);
?>
```

PHP CAPTCHA

Create CAPTCHA image PHP

A CAPTCHA is a type of challenge-response test used in computing as an attempt to ensure that the response is generated by a human being. It is an acronym based on the word "capture" and standing for "Completely Automated Public Turing test to tell Computers and Humans Apart.

Create Arithmetic CAPTCHA

```
<?php

error_reporting(1);

$arr=range(99,9);

$brr=range(99,9);
```

Ashwani Mishra

```
$randa=array_rand($arr);

$randb=array_rand($brr);

$a=$arr[$randa];

$b=$brr[$randb];

$r=$a+$b;

$cap=$a."+".$b;

if(isset($_POST['b1']))

 {

   if($_POST['t2']==$_POST['t3'])

   {

   echo '<center>'.'<font color="blue" size="5">'."Welcome user".'</font>'.'</center>';

   }

  else

  {

  echo '<center>'.'<font color="red" size="5">'."Please fill the correct
  answer".'</font>'.'</center>';

  }

}

?>


  <html>

   <center>

   <form method="post">

<?php

error_reporting(1

); echo $cap."=";
```

```
?>

<input type="hidden" name="t3" value="<?php echo $r; ?>">

<input type="text" name="t2" autofocus><br>

<input type="submit" name="b1" value="match">

</form>

</center>

</html>
```

In the above example First create a variable $arr with multiple values i.e created by range function(using array range( ) function). second variable $brr also hold multiple values between "9" to "99". Now pass $arr inside array_rand( ) function, it returns a random array index which stores in
$randa and $randb variable. Now call the value of array $arr[$randa] and store it in $a and $b variable. In next line create a variable $cap which holds a string ($a."+".$b) print the $cap variable it shows two arithmetic numbers seperated by "+". Now calculate the value, enter the calculated value in text-box, to check the addition of two numbers is correct or not. if addition of two numbers entered by user is right it shows a message "Welcome user" otherwise show error message "Please fill the correct answer".

PHP Create CAPTCHA Security Images

```
<form action="" method="post" >

  <table border="0" align="center">

    <tr>

      <td width="215"> Validation code:</td>

      <td width="162">

<?php

$arr= array_merge(range(0,9),range("A","Z"));

//print_r($arr);

for($i=1;$i<=5;$i++)

{
            $ch = $arr[array_rand($arr)];

            @$captcha=$captcha.$ch;

            @$fc=$fc.$ch.",";
```

```
}

//echo $fc."<br>";
$nar = explode(",",$fc);

for($i=0;$i<5;$i++)

{

echo $nar[$i];
//echo "<img src='$nar[$i].GIF'/>";

}

if(isset($_POST['match']))

{

   if($_POST['img']==$_POST['hid'])

    {

     echo  "<br/><font color='blue'>security code matched</font>";

    }

   else

   {

            echo "<br/><font color='red'> try again</font>";

   }

}

?>

   <tr>

       <td>Enter the  above code here :</td>

      <td> <input name="img" type="text">

       </td>

   </tr>

   <tr>
```

```
    <td colspan="2" align="center">

            <input name="match" type="submit"  value="Submit Security code"></td>

   </tr>

  </table>

<input type="hidden" value="<?php echo $captcha; ?>" name="hid"/>

</form>

</pre>

</div><br/>
```

Output

PHP Exception Handling

Exceptions are used to change the normal flow of a script if a specified error occurs.

What is an Exception

With PHP 5 came a new object oriented way of dealing with errors.

Exception handling is used to change the normal flow of the code execution if a specified error (exceptional) condition occurs. This condition is called an exception.

This is what normally happens when an exception is triggered:

- ***The current code state is saved***
- ***The code execution will switch to a predefined (custom) exception handler function***
- ***Depending on the situation, the handler may then resume the execution from the saved code state, terminate the script execution or continue the script from a different location in the code***

We will show different error handling methods:

- ***Basic use of Exceptions***
- ***Creating a custom exception handler***
- ***Multiple exceptions***
- ***Re-throwing an exception***

- *Setting a top level exception handler*

Note: Exceptions should only be used with error conditions, and should not be used to jump to another place in the code at a specified point.

Basic Use of Exceptions

When an exception is thrown, the code following it will not be executed, and PHP will try to find the matching "catch" block.

If an exception is not caught, a fatal error will be issued with an "Uncaught Exception"

message. Lets try to throw an exception without catching it:

```
<?php
//create function with an
exception function
checkNum($number) {
if($number>1) {
   throw new Exception("Value must be 1 or below");
 }
 return  true;
}

//trigger
exception
checkNum(2);
?>
```

The code above will get an error like this:

```
Fatal error: Uncaught exception 'Exception'
with message 'Value must be 1 or below' in
C:\webfolder\test.php:6 Stack trace: #0
C:\webfolder\test.php(12):
checkNum(28) #1 {main} thrown in C:\webfolder\test.php on line 6
```

Try, throw and catch

To avoid the error from the example above, we need to create the proper code to handle an

exception. Proper exception code should include:

1. ***try - A function using an exception should be in a "try" block. If the exception does not trigger, the code will continue as normal. However if the exception triggers, an exception is "thrown"***
2. ***throw - This is how you trigger an exception. Each "throw" must have at least one "catch"***
3. ***catch - A "catch" block retrieves an exception and creates an object containing the exception information***

Ashwani Mishra

Lets try to trigger an exception with valid code:

Ashwani Mishra

```php
<?php
//create function with an
exception function
checkNum($number) {
if($number>1) {
   throw new Exception("Value must be 1 or below");
  }
  return  true;
}

//trigger exception in a "try"
block try {
  checkNum(2);
  //If the exception is thrown, this text will not
  be shown echo 'If you see this, the number is
  1 or below';
}

//catch exception
catch(Exception $e) {
  echo  'Message:  ' .$e->getMessage();
}
?>
```

The code above will get an error like this:

Message: Value must be 1 or below

Example explained:

The code above throws an exception and catches it:

1. ***The checkNum() function is created. It checks if a number is greater than 1. If it is, an exception is thrown***
2. ***The checkNum() function is called in a "try" block***
3. ***The exception within the checkNum() function is thrown***
4. ***The "catch" block retrieves the exception and creates an object ($e) containing the exception information***
5. ***The error message from the exception is echoed by calling $e->getMessage() from the exception object***

However, one way to get around the "every throw must have a catch" rule is to set a top level exception handler to handle errors that slip through.

Abhwani Mishra